Java Core Concepts

Comprehensive Analysis of 20 Technical Programming Questions



Q1: Method Overloading Priority

The Code

```
int x = 23; int y = 10;
method(x, y);

// Definitions:
method(String a, int b) { ... }
method(int a, int b) { ... }
method(double a, int b) { ... }
```

ANSWER: METHOD 2

Technical Logic

In Java, the compiler searches for the most specific match first. Since **x** and **y** are both `int`, Method 2 is an **exact signature match**.

- Method 1 fails (Type mismatch).
- Method 3 is a candidate (widening), but priority goes to the exact match.

Q2: Selection Sort Complexity

Algorithm Context

Target Array: `{8, 3, 1, 2, 6}`

The algorithm iterates through the array *n-1* times, finding the minimum element and performing a swap at the end of each outer loop iteration.

ANSWER: 4 SWAPS

Tracing the Passes

For an array of length 5, the outer loop runs exactly 4 times (0 to length-2).

1. Pass 1: Min found at index 2 (1). Swap 8 & 1.
2. Pass 2: Min found at index 3 (2). Swap 3 & 2.
3. Pass 3: Min found at index 3 (3). Swap 8 & 3.
4. Pass 4: Min found at index 4 (6). Swap 8 & 6.

Q3: Arithmetic Type Truncation

The Code

```
int x = 10;
int y = 5;
System.out.println(x / y);
```



ANSWER: 2

Why not 2.0?

In Java, **Integer Division** occurs when both operands are `int`. The result is strictly an `int`.

For the output to be `2.0`, at least one operand would need to be a `double` or `float`.

Q4: Array Indexing Syntax

CORRECT SYNTAX: array[1] = 20;

The Logic of Zero-Based Indexing

In Java, arrays are zero-indexed. To target the **second position**, we must use index 1.

```
int[] array = new int[10];
array[0] = 10; // 1st Position
array[1] = 20; // 2nd Position
```

Q5: Boolean Evaluation

```
int x = 8;
System.out.println(x == 8);
```

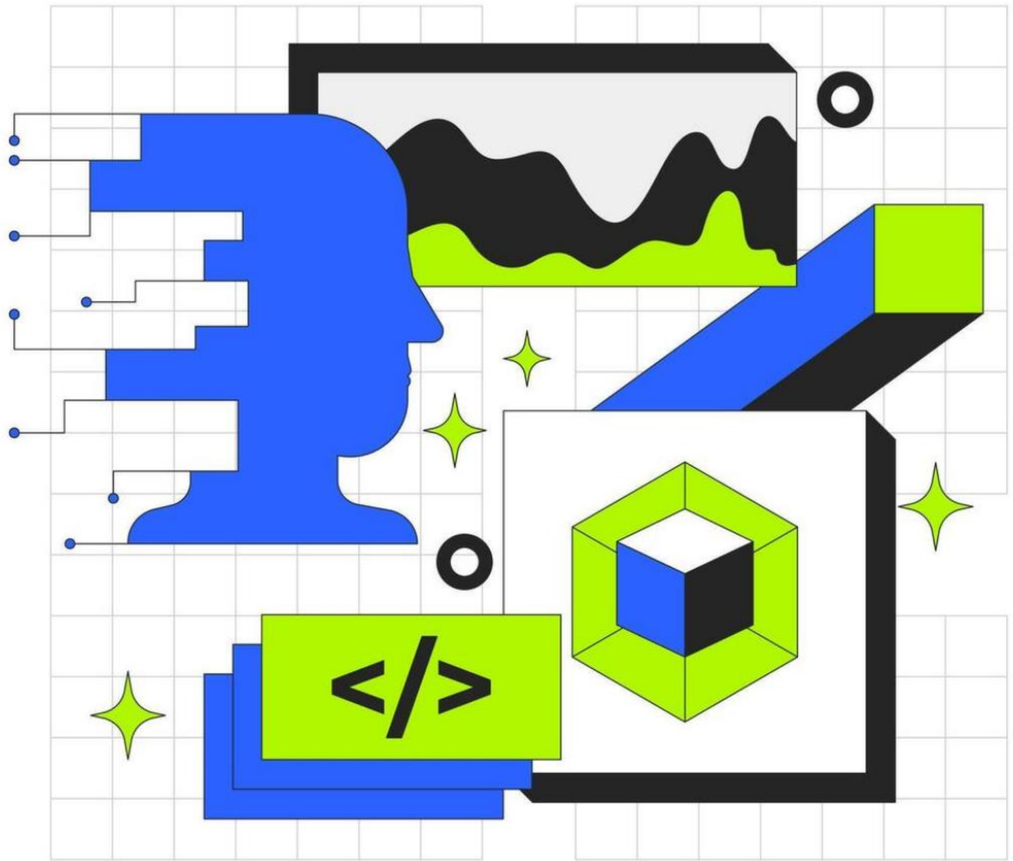
ANSWER: TRUE

Assignment vs Comparison

- = is the **assignment operator**. It sets a value.
- == is the **relational operator**. It returns a boolean (true/false) by checking equality.

Since x is indeed 8, the expression evaluates to `true`.

Q6: Understanding Constants



DEFINITION: A VALUE THAT CANNOT CHANGE

The 'final' Keyword

In Java, constants are declared using the `final` keyword. This tells the JVM that the memory location is read-only after the initial assignment.

```
final int MAX_SPEED = 120;
```

Q7: Sequential Logic Trace

```
int var_1 = 1;
int var_2 = var_1 + 2; // 3
int var_3 = var_1 + var_2; // 1 + 3
```

ANSWER: 4

The Path of Execution

1. var_1 is 1.
2. var_2 evaluates to 3.
3. var_3 adds 1 and 3 to yield 4.

Q8: Argument Mapping

Method Signature

```
myMethod(int b, int a)
```

The Call

```
myMethod(a, b); // where a=44, b=5
```

ANSWER: 5

Variable Shadowing Trick

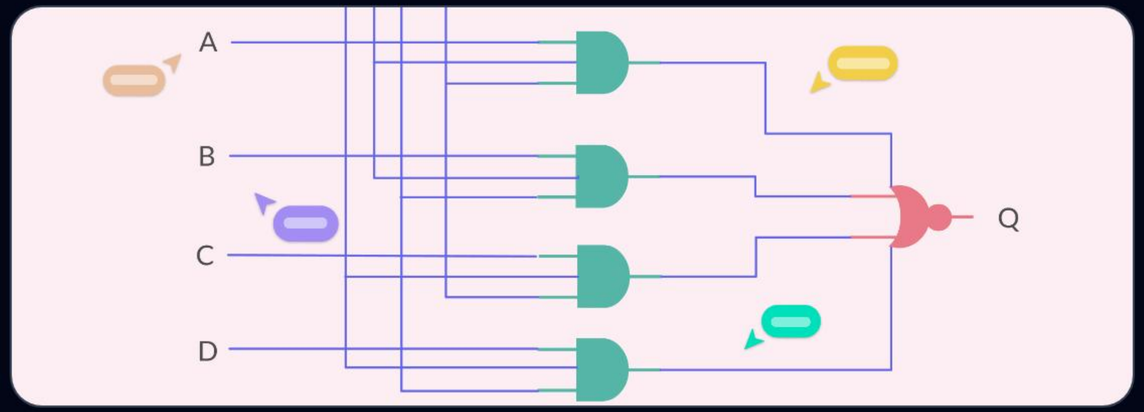
Java parameters are positional. Even though the names are swapped:

- Argument 1 (a=44) maps to Parameter b.
- Argument 2 (b=5) maps to Parameter a.

The method returns a, which holds 5.

Q9: Short-Circuit Logic

```
boolean submit = false;
if (!submit) { ... }
```



OUTPUT: I WILL NOT PASS THE MODULE.

Evaluation Flow

The program enters the `else if (!submit_coursework)` block because `submit_coursework` is false, making `!false` evaluate to true.

Q11: Valid Initialization

CORRECT: int[] arr = {1, 2, 3};

Syntactic Sugar

This "Array Initializer" shorthand is only valid during the declaration of the variable. It automatically manages memory allocation and size definition based on the elements provided.

Q12: Cumulative Logic Trace

```
x = 5; y = 2;
x = x + y; // x becomes 7
y = y + 3; // y becomes 5
z = x + y; // 7 + 5
```

ANSWER: 12

The Result

Variables are updated in-place. By the time z is calculated, both x and y have evolved from their initial states.

Q13: Exiting Iteration



KEYWORD: BREAK;

Functionality

The `break` statement terminates the innermost loop or switch statement immediately. Control flows to the statement following the loop body.

Q14: Multi-Dimensional Access

```
double[][] arr = {
  {23.1, 34.4, ...},
  {32.8, 67.5, 56.0, ...}
};
```

TARGET: arr[1][2]

Grid Navigation

- Row 1 (Index 0): Top row.
- Row 2 (Index 1): Contains 56.0.
- Column Index 2: The 3rd element in that row.

Q15: Primitive Data Types

Classification

400000 → **int**

true → **boolean**

25.64d → **double**

Classification

"Text" → **String**

8.79f → **float**

Q16: Incremental Output

```
for (int i = 3; i ⩽ 12; i = i + 3)
```

OUTPUT: 3, 6, 9, 12,

Execution Steps

The loop starts at 3 and adds 3 at each step. Because the condition is <= 12, the number 12 is included in the output before the condition fails at 15.

Q17: Calling Overloaded Methods

Variable: `String message`

Method needed: One that takes a `String`.

CALL: myMethod(message)

Resolution

Java matches the `String` argument to the method signature `myMethod(String message)`, ensuring the correct block of code prints the dynamic text.

Q18: Array Lookup

```
array = {0, 30, 45, 78, 8};
```

ANSWER: 78

The Index Map

Target: `array[3]`

- Index 0: 0
- Index 1: 30
- Index 2: 45
- **Index 3: 78**

Q19: Double Addition

```
double x = 12.0;
double y = 4.0;
return a + b;
```

ANSWER: 16.0

Precision Matters

Because the return type and inputs are `double`, the decimal `.0` must be maintained in the output representation.

Q20: The Remainder Operator

```
System.out.println(15 % 4);
```

$$15 = (4 \times 3) + 3$$

ANSWER: 3

Mathematical Basis

The Modulo operator (`%`) calculates the remainder of an integer division. 4 goes into 15 three times, with **3 left over**.

Mastery Achieved

Review complete. All 20 core concepts have been analyzed.

End of Presentation

Image Sources



https://miro.medium.com/v2/resize:fit:1400/0*4d4_meQox5rWClC7

Source: levelup.gitconnected.com



https://static.vecteezy.com/system/resources/previews/071/988/358/non_2x/web-development-2d-onboarding-ui-illustration-tech-driven-algorithm-coding-artificial-intelligence-and-data-science-programming-cartoon-concept-isolated-on-white-flat-abstract-metaphor-vector.jpg

Source: www.vecteezy.com



https://creately.com/static/assets/guides/logic-gate-diagram/logic-gate-diagram-hero.webp

Source: creately.com



https://png.pngtree.com/thumb_back/fw800/background/20221015/pngtree-actual-screen-displaying-java-script-code-development-programming-workflow-in-action-photo-image_28457682.jpg

Source: pngtree.com